

## DAG Pipeline Declared by pipeline.yaml |

Single-project pipelines read `infrastructure/core/pipeline/pipeline.yaml`. `scripts/execute_pipeline.py` expands the declarative DAG, applies tag filters (`--core-only` skips llm stages), checkpoints between nodes, then dispatches numbered scripts (`scripts/NN_*.py`) or builtin methods (`_run_clean_outputs`).

The **default YAML graph contains ten named stages** (plus telemetry configuration metadata):

1. **Clean Output Directories** — wipes prior `projects/<name>/output/` + delivered `output/<name>/` paths so stale PDFs cannot satisfy validation.
2. **Environment Setup** (`00_setup_environment.py`) — Python/uv probing, toolchain discovery, scaffolding directories, `PYTHONPATH` wiring.
3. **Infrastructure Tests** (`01_run_tests.py --infra-only`) — tests/ suite with infra coverage thresholds ( 60 %).

## DAG Pipeline Declared by pipeline.yaml II

4. **Project Tests** (01\_run\_tests.py --project-only) — per-project suites with 90 % coverage mandate.
5. **Project Analysis** (02\_run\_analysis.py) — lexicographically ordered projects/<name>/scripts/\*.py, each a thin orchestrator (src/ does real work).
6. **PDF Rendering** (03\_render\_pdf.py) — Pandoc → XeLaTeX loop, bibliography assembly, injected variables from Stage 02 artefacts.
7. **Output Validation** (04\_validate\_output.py) — PDF structure, manifests, Markdown hygiene.
8. **LLM Scientific Review** (06\_llm\_review.py --reviews-only; tags: llm) — executive + quality critiques via local Ollama; allow\_skip: true.
9. **LLM Translations** (06\_llm\_review.py --translations-only; tags llm, same dependency edges) — multilingual abstract expansion.

## DAG Pipeline Declared by `pipeline.yaml` III

10. **Copy Outputs** (`05_copy_outputs.py`) — reproducible snapshots into canonical `output/<project>/`.

Two LLM nodes intentionally share one script module with orthogonal CLI switches; both depend only on validation so they can parallelize logically while remaining optional.

**Executive reporting** (`scripts/07_generate_executive_report.py`) is **not** a YAML node inside the single-project executor. `--all-projects / execute_multi_project.py` invokes it once after iterating projects, consolidating cross-project KPIs dashboards.

Topological order therefore differs slightly from lexical script numbering (e.g., `copy` executes after `validation` even though script 05 precedes 06 lexically).

# DAG Pipeline Declared by `pipeline.yaml` IV

## Stage Highlights

**Infrastructure vs project tests.** Splitting pytest invocations isolates flaky infra regressions (`MAX_TEST_FAILURES` knobs) from zero-tolerance gates on domain code (`max_project_test_failures` default 0 declared in YAML front-matter/testing blocks).

**Stage 02 illustration.** The analysis stage is deliberately concrete rather than a hypothetical diagram factory: each canonical project ships real behaviour at this node. `template_autoresearch_project` runs readiness validation; `template_search_project` merges remote literature JSON, generates scripted figures (`y_generate_search_figures.py`), and writes manifests; `template_code_project` emits optimization plots; and `template_prose_project` triggers structural validation scaffolding. The pipeline shape is identical across all four—only the Stage 02 payload differs—which is exactly what lets one orchestrator serve heterogeneous research domains.

# DAG Pipeline Declared by pipeline.yaml V

## Interactive Orchestration

### run.sh

Thin wrapper invoking `python -m infrastructure.orchestration`. Offers:

- ▶ per-project staged execution,
- ▶ chained digits (234 shorthand),
- ▶ multi-project grid (a–d presets),
- ▶ graceful quit / resume parity with `scripts/README.md`.

Selecting **d alone** after a passing multi-project run exits immediately once summaries print—avoiding repetitive menu redraw.

### secure\_run.sh

Executes Python secure path: standard pipeline artefact reproduction **then** invokes `run_secure_pipeline` for steganographic PDF hardening (`infrastructure.steganography`). Original PDFs stay immutable; hardened companions carry QR overlays plus hash manifests sidecars.